Assignment type: **Assignment 1 -Cryptocurrency and Digital Assets**

Submission type: **PDF**

Github link : https://github.com/Songa7273/Assignment-1--Cryptocurrency-and-Digital-Assets

Video Link: assignment 1.mp4

Deadline: **1st November 2026**

1.Abstract

The aim of this project is to show the functions of cryptocurrency exchanges that enable the users to display,purchase, sell and keep the digital assets safe. In the current project, the functionality of Ethereum smart contracts is illustrated through the process of registering a real-life digital asset. ALUAssetRegistry contract creates a unique ERC-721 token having the metadata of the ALU logo along with the unique SHA-256 fingerprint, and ALULogoToken is responsible for issuing limited ERC-20 tokens based on the previously mentioned ERC-721 token.

2. Introduction

It is hard to tell if the image file is original or edited because it is possible to replicate images and make alterations easily.

With the help of a blockchain-based ledger, it is possible to create the record of the image file fingerprint in the public domain, indicating when and where it was registered.

The SHA-256 hash here serves as the unique fingerprint of the image file; any further change in the image will lead to a different hash. Thus, a blockchain does not prevent the person from making copies of the image; meanwhile, using a hash does not mean somebody owns the copyright to the file.

The project uses two types of contracts for different purposes. ALUAssetRegistry uses ERC-721 as the registered logo is treated as a unique asset having its own token ID and metadata, while ALULogoToken operates under ERC-20 because the shares issued are interchangeable tokens.

3. Question 1- Exploring a Cryptocurrency Exchange

3.1 Account creation and identity verification

I investigated Coinbase, a company that deals with trading cryptocurrencies. During the signing up process the system instructed me to provide an email address, choose a password, verify the phone number, fill out my full name, confirm my main country of residence and

provide additional information as the system requested during the registration process. Based on the kind of user's account and location the system may also require to submit an identity document issued by the governmental authorities and verify it through sending a photo of a user's face or doing any of the verification steps mentioned above. All processes of verifying an identity are used for lowering the risks of fraud, making sure that all methods of payment belong to the accountholder and fulfilling all customer identity verification obligations and anti-money laundering regulations.

3.2 Platform sections and cryptocurrencies

The Coinbase platform comprises different areas for browsing digital currencies, checking account balances and portfolio data as well as buying, selling, and trading. The examined cryptocurrencies were Bitcoin (BTC), Ether (ETH), and Solana (SOL). Bitcoin is a decentralised digital form of money limited to 21 million units. Ether is a digital currency created for the Ethereum network that pays for transaction costs and permits a user to use the available apps. Solana plays a similar role as Ether but is specific for the Solana blockchain.

3.3 Payment methods

The payment-methods feature of Coinbase allows users to link any accepted payments like bank transfers and debit card payments, thus depositing money to start trading cryptocurrencies. However, which payment methods are generally allowed and accepted depends on whether some of the payments are legal in the user's country. As I was checking out the payment-methods feature in my country (Rwanda), no information was available to let me know what payment methods I could use along with their associated fees and settlement times.

In general, debit cards provide faster transactions in comparison to bank transfer which might take longer based on the bank's processing system. However, regarding the costs and time needed to process different methods, this is usually very country specific. Hence, I could not directly compare those two methods through my account with Coinbase.

3.4 Simulated purchase of $100

A market order is meant to be carried out promptly at the best possible market price, but the price might differ. A limit order allows the buyer to indicate the maximum price they would be willing to pay, with the possibility that the order might not be filled, if the market price does not arrive at that level.

In this project, I used Bitcoin (BTC) as an example, planning to buy it for a total budget of $100. Using the example Bitcoin price of $77,703.31 per BTC, I calculated that I would incur a transaction fee of $1.50. This would give me a total of $98.50 for the purchase, which translates to approximately 0.00126764 BTC in my purchase.

I've used a market order due to the fact that it is created to be executed at the current market price, which is why I used it in this simple purchase scenario. The numbers used are not actual figures from the transaction made but only the estimates as I did not access Coinbase successfully to use the platform.

3.5 Transaction records and transaction hashes

The order history or transaction history of an exchange can indicate details such as the asset involved, order type, buy/sell directions, amount, cost, fees, time when a certain order was placed, and status. Transaction hash is a unique code that identifies the blockchain transaction. Whenever the transaction occurs on-chain, we can check its information using the block explorer. However, not every movement registered within the centralized exchange means a separate on-chain transaction. Therefore, the internal reference code can differ from the hash identifier of the blockchain transaction.

3.6 Security features

Coinbase has different methods of its two-step verification feature such as SMS verification, authenticator apps, and security keys. However, options will differ depending on the types of accounts and regions. With two-step verification, Coinbase offers extra protection beyond the password, making it more difficult for any unauthorized access person to enter the account. In addition, the company provides security measures on the accounts and transaction limitations which vary depending on the account, region, method of payment, and transaction background.

The two security measures mentioned are 2-step verification and transaction limitations. Both these measures are necessary because transactions made in cryptocurrency mostly cannot be reversed once verified. Thus, ensuring restricted access to the account and monitored transaction operations protects the user from unneeded transfer and loss of money.

4. Blockchain Setup

The development tool Hardhat is used for compiling, testing, and deploying smart contracts on the Ethereum platform. The local blockchain refers to the local Ethereum network established on a developer's computer. The local Ethereum network provides access to practice accounts and Ethereum currency, enabling contract deployment and operation without additional costs or actual deployment on the actual Ethereum. The blockchain account's wallet address is the public address of the blockchain account. The blockchain wallet address is used to receive funds and interact with smart contracts, yet it should be distinguished from the secret key, which is used during the transaction authorization.

For this assignment, we use Node.js and npm as a JavaScript runtime and the package management tool. The Hardhat tool compiles Solidity contracts, while the testing tool operates locally. The project is supported by the OpenZeppelin Contracts library, which includes working ERC-721, ERC-20, and ownership models that spare developers from programming the standard tokens from the very beginning.

5. Smart Contract Explanation

5.1 ALU Asset Registry (ERC-721)

ALU Asset Registry certifies ALU logo as a special non-fungible token. The AssetMetadata structure keeps the information on the asset name and its file type, SHA-256 content hash, wallet address for registering, and the date and time when registration occurred in Unix format. One mapping keeps the information on all the token IDs and corresponding metadata, and the other keeps the information on whether a particular content hash was already registered. This way, the file fingerprint can never be registered for the second time within that smart contract. The registerAsset() function ensures whether the specified hash is used or not, increases the token count, mints an NFT for the user through _safeMint(), saves the metadata in the smart contract, marks the hash as used, generates the AssetRegistered event, and returns the token ID. The verifyLogoIntegrity() function compares the given hash with the stored hash and returns the result of authenticity and the message. The getAsset() function returns the metadata related to the token ID. Since the checking function only needs to read the stored information, it does not involve any on-chain transaction or gas fees.

5.2 Logo hash

The SHA-256 hash must be calculated from the exact ALU logo image file included with the submission. It must not be a placeholder. If the image file is changed, even by resizing or converting it, its bytes change and the resulting hash will normally change as well.

5.3 ALULogoToken (ERC-20)

The ALULogoToken issued by the ALU Logo Token (abbreviated as ALUT) erases a surplus of 1,0000,000 coins of specified identity and value. The constructor will send all coins to the wallet of the user whose address is indicated in logoOwner. The function distributeShares() is available only for the owner of the platform. The function checks whether the amount is more than zero and moves funds from the balance of the owner to the designated user.
Ownership Percentage is a view function that allows calculating the percentage of ownership of the particular token holder at a given time.
In a more practical sense, since tokenization means issuing tokens that show the possession of property or any interest in it, in this scenario ALUT token only provides indirect legal rights for the ownership of ALU trademark.

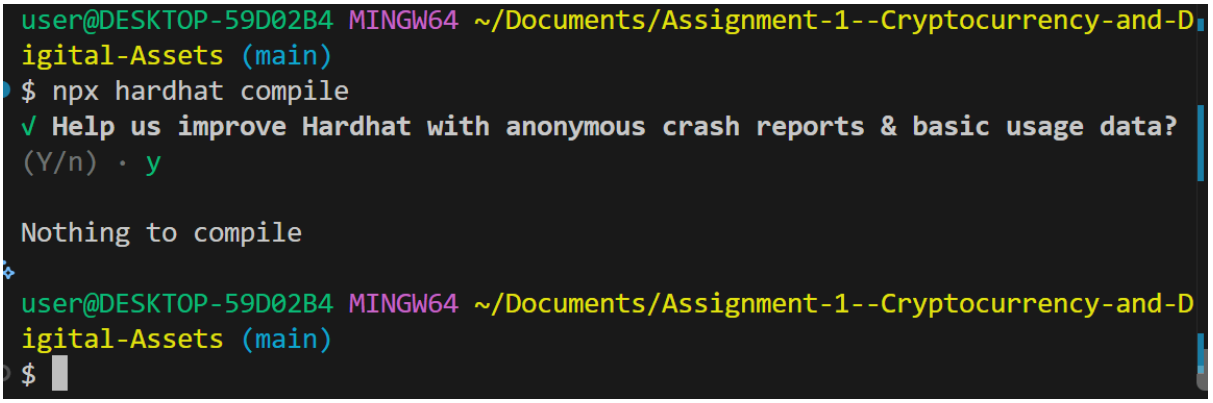
6.Testing and deployment

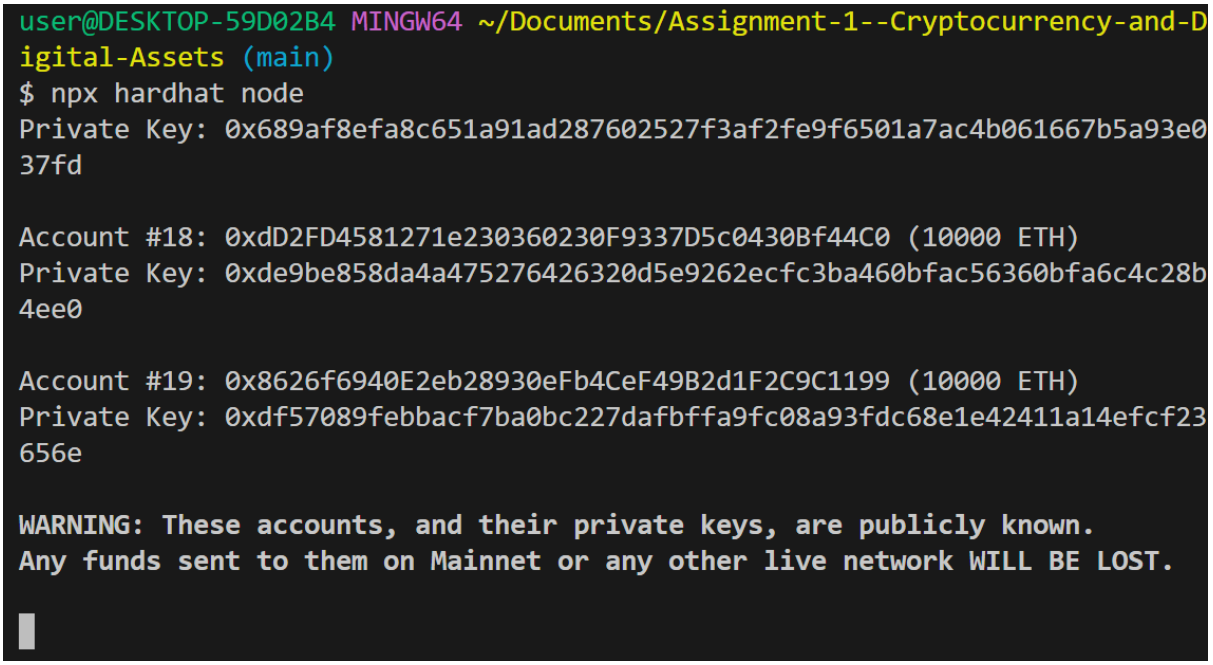
The implementation and examination of smart contracts on a local blockchain network are accomplished using Hardhat and Visual Studio Code.

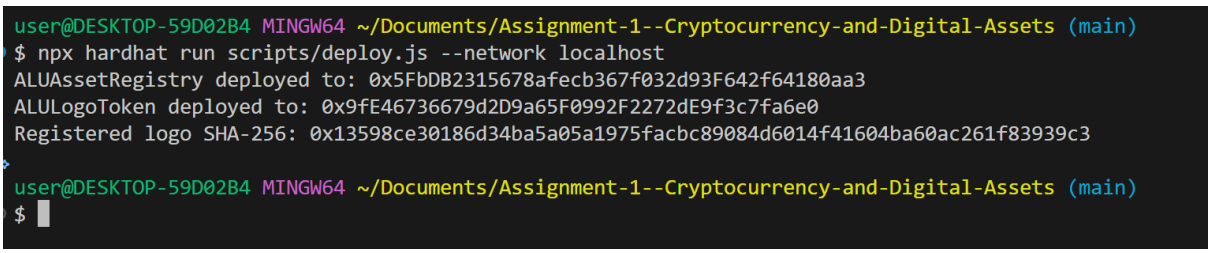
The execution of ALU Logo Token testing validated that the token was generated with the appropriate name (ALU Logo Token), mentioned symbol (ALUT), and specified overall supply of 1 million units. The transaction of 100 units of ALUT from one wallet account to another was performed without any issue, and any updates in recipient's and owner's wallets were accounted for.

The scrutiny of the ALU Asset Registry proved the registration of ALU Logo, the existence of required metadata, as well SHA-256 content hash acquisition and validation of ownership/provenance was completed satisfactorily.

Results of this experimentation indicate that both smart contracts can run on a local machine as intended.







```
user@DESKTOP-59D02B4 MINGW64 ~/Documents/Assignment-1--Cryptocurrency-and-Digital-Assets (main)
$ npx hardhat console --network localhost
Welcome to Node.js v22.14.0.
Type ".help" for more information.
> const [owner, recipient] = await ethers.getSigners()
undefined
> const token = await ethers.getContractAt(
...   "ALULogoToken",
...   "0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0"
... )
undefined
> await token.name()
'     ogo Token'
> await token.symbol()
'   T'
> ethers.formatUnits(await token.totalSupply(), 18)
'1000000.0'
> ethers.formatUnits(await token.balanceOf(owner.address), 18)
'1000000.0'
>
```

```
$ npx hardhat console --network localhost
'1000000.0'
> ethers.formatUnits(await token.balanceOf(owner.address), 18)
'1000000.0'
> const tx = await token.connect(owner).transfer(
...   recipient.address,
...   ethers.parseUnits("100", 18)
... )
undefined
> await tx.wait()
ContractTransactionReceipt {
  provider: HardhatEthersProvider {
    _hardhatProvider: LazyInitializationProviderAdapter {
      _providerFactory:     n   n  i n (an n m   ) ,
      provider:   a   a     m a i i i     i     a     ,
      _emitter:     n  mi     ,
      _initializingPromise:     mi
    },
    _networkName: 'localhost',
    _blockListeners: [],
    _transactionHashListeners: Map(0) {},
    _eventListeners: []
  },
  to: '0 9 E46736679d2D9a65 0992 2272dE9 3c7 a6e0',
  from: '0  39 d6e51aad88 6 4ce6aB8827279c   b92266',
  contractAddress: null,
  hash: '0 0ba2043d798e0e 6d31d 3168a8c9a177 13 108c407b5e52c5e075ab403ce1e',
  index: 0,
  blockHash: '0 ed909142189629734 4dcabeb1683503 7abea6ae63c47d24b0b87c73473c4d5',
  blockNumber: 4,
  logsBloom: '0 00000000000000000002000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000040200000
00000000000000000000000000000000000000800000000000000000000000000000000000000000000000000000000000000000000000000000000000000010
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000002000002000000000000000000000002000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
',
  gasUsed: 52245n,
  blobGasUsed: undefined,
  cumulativeGasUsed: 52245n,
  gasPrice: 756502705n,
```

```
    gasPrice: 756502705n,
    blobGasPrice: undefined,
    type: 2,
    status: 1,
    root: undefined
  }
  > ethers.formatUnits(await token.balanceOf(recipient.address), 18)
  '100.0'
  > ethers.formatUnits(await token.balanceOf(owner.address), 18)
  '999900.0'
  >
```

```
> const registry = await ethers.getContractAt(
...   "ALUAssetRegistry",
...   "0x5FbDB2315678afecb367f032d93F642f64180aa3"
... )
undefined
> await registry.hashRegistered(
...   "0x13598ce30186d34ba5a05a1975facbc89084d6014f41604ba60ac261f83939c3"
... )
true
> await registry.verifyLogoIntegrity(
...   1,
...   "0x13598ce30186d34ba5a05a1975facbc89084d6014f41604ba60ac261f83939c3"
... )
Result(2) [ true, ' ogo is authentic.' ]
> await registry.getAsset(1)
Result(5) [
  '    O  icial  ogo',
  'png',
  '0 13598ce30186d34ba5a05a1975 acbc89084d6014 41604ba60ac261 83939c3',
  '0  39 d6e51aad88 6 4ce6aB8827279c   b92266',
  1793372471n
]
> await registry.ownerOf(1)
'0  39 d6e51aad88 6 4ce6aB8827279c   b92266'
>
```

7. Automated Test Results

The test suite contains eight automated tests. The run reported “8 passing (5s)”. These tests check both the registry and tokenisation behaviour.

Test	What it verifies
1. Successful registration	A new logo hash can be registered and a token ID is returned.
2. Duplicate hash rejection	The contract rejects a second registration using the same hash.
3. Correct hash verification	The authenticity check returns true and the expected message for the registered hash.
4. Incorrect hash verification	The check returns false and the warning message for a different hash.
5. Metadata retrieval	getAsset() returns the expected asset name and file type.
6. Fixed token supply	The full 1,000,000 ALUT supply is minted

	to the logo owner.
7. Share distribution	The owner can transfer the specified amount to a recipient.
8. Ownership percentage	The function returns the expected whole-number percentage for a known balance.

```
user@DESKTOP-59D02B4 MINGW64 ~/Documents/Assignment-1--Cryptocurrency-and-
igital-Assets (main)
$ npx hardhat test

  ALU logo contracts
    ✓ registers the ALU logo and returns a token ID (2 2ms)
    ✓ rejects a second registration with the same hash (  ms)
    ✓ returns true and the authentic message for the correct hash
    ✓ returns false and a warning for an incorrect hash
    ✓ returns the registered asset metadata (43ms)
    ✓ mints the full fixed supply to the logo owner
    ✓ allows the owner to distribute shares
    ✓ calculates whole-number ownership percentages


  8 passing ( s)
```

7. Conclusion

This project illustrates the potential of blockchain technology as a medium to document the entrusted ownership of digital files, as well as functioning of various token standards, which support varied asset models. A better choice for one-off logo registration is ERC-721, where every unique NFT has its own token ID and metadata. On the other hand, ERC-20 was a more suitable option for fungible shares of ALUT tokens, since every ALUT token is interchangeable with each other. The tests confirm that all core functions work as intended in the local development environment.

Were the project to be further expanded, opportunities for improvement would include launching experiments using Ethereum test network, confirming validity of the source of a launched contract, additional testing and reinforcement of access controls, and developing the web interface expected to be completed in the next project. In addition to the above, any real tokenization of the university intellectual property would also require relevant legal contracts determining what the token owners can own.

9. References

• Coinbase, “New customer resource guide”:https://help.coinbase.com/en/new-customer-guide

• Coinbase, “Verify your identity on Coinbase”:https://help.coinbase.com/en/coinbase/getting-started/getting-started-with-coinbase/id-doc-verification

• Coinbase, “Order management”: https://help.coinbase.com/en/coinbase/trading-and-funding/advanced-trade/order-managemet

• Coinbase, “2-Step Verification Setup”:https://help.coinbase.com/en/exchange/managing-my-account/2fa-setup

• OpenZeppelin Contracts documentation: https://docs.openzeppelin.com/contracts/

• Hardhat documentation: https://hardhat.org/docs